

Metrics that Matter

A Layered Measurement Strategy for Product, Platform, and Delivery Signals

1. Purpose and Principles

This strategy establishes a common measurement architecture across the software delivery organization. It defines three distinct but linked layers of signal: what users do (product analytics), how the product behaves (technical telemetry), and how the organization delivers (programmatic metrics). Revenue and conversion measures are deliberately out of scope; this framework measures experience, reliability, and flow.

Five principles govern collection and use of every metric in this document:

Metrics are system health indicators, not scorecards. No metric in this framework is used to rank individuals or teams. Programmatic metrics in particular are diagnostic instruments for improving the delivery system.

Segment before you aggregate. Aggregate numbers hide failure. Every technical metric is collected per endpoint, per service, and per client cohort, then rolled up. An aggregate 2% error rate can conceal one endpoint failing 40% of the time.

Trend deltas, not absolutes. Most signals carry a normal baseline. The actionable event is a step change, especially one correlated with a deployment, release, or policy change.

Alert on what pages people; trend the rest. Only SLO-bound reliability signals wake engineers. Everything else is reviewed on a defined cadence.

Baseline each converged organization separately. Legacy organizations enter with different tooling, pipelines, and debt. Org-wide targets are set only after per-organization baselines are established, so early targets do not punish teams that inherited the weakest pipelines.

2. The Three-Layer Architecture

Each layer answers a different question and has a different owner, cadence, and toolchain. The layers are collected independently but joined through shared dimensions (release version, service, endpoint, user cohort) so that a signal in one layer can be traced to its cause or consequence in another.

Layer	Question Answered	Primary Owner	Review Cadence
1. Product Analytics	What are users doing, and are they succeeding?	Product and UX	Weekly; cohort reviews monthly
2. Technical Telemetry	How is the product behaving in production?	Engineering and SRE	Real-time alerting; weekly review
3. Programmatic Delivery	How effectively does the organization deliver change?	Engineering leadership	Per sprint; quarterly baseline review

3. Reference Toolchain and Ownership Model

The strategy is implemented on four systems already in place, with no external analytics SaaS. Product analytics is implemented on Elastic rather than a commercial behavioral-analytics product, using a shared client event library and dedicated product-events indices.

System	Role in the Strategy
Elastic	Observability backbone: APM (transactions, spans, deployment annotations), RUM (Core Web Vitals, JS errors, sessions), infrastructure metrics via Elastic Agent, Synthetics/Heartbeat, ML anomaly detection, SLO definitions with burn-rate alerting, and Kibana dashboards. Also hosts the product-events, deployments, and PR-analytics indices.
CI/CD pipeline	Source of delivery telemetry: emits deployment events (service, version, commit SHAs, timestamps), posts APM deployment annotations, publishes coverage and build/test results, and auto-creates ServiceNow change records on production deploys.
Jira	Work-item system of record: work-type taxonomy for the work ratio, escaped-defect flagging, board data for WIP/throughput and cumulative flow.
ServiceNow	Incident and change system of record: incident timestamps for MTTD/MTTR, change records linked to deployments for change failure rate and recovery time, support tickets and CSAT for the support signals, scheduled export into Elastic for joins.

Ownership model

Every metric in Sections 4 through 9 carries one of three ownership tags:

Central. Built and operated once by platform engineering; app teams consume it with no per-team work beyond adopting the paved road.

Shared. Platform engineering provides the capability (library, pipeline stage, dashboard template, ML job, integration); each app team supplies the semantics: instrumentation, definitions, thresholds, or triage discipline. Most metrics land here, and the split within each is stated in the implementation column.

App. Primarily each application team's responsibility, with platform engineering providing only ingestion.

The paved road

Platform engineering builds once: APM and Elastic Agent baked into base images and golden pipelines so services are instrumented by default; the shared client event library (product events, friction detection, RUM snippet, in-app survey component); index templates and ingest pipelines enforcing the shared dimension schema (release version, service, endpoint, anonymized cohort); deployment-event emission and APM deployment annotations as pipeline stages; automated ServiceNow change-record creation; ML anomaly jobs and SLO/burn-rate alert rule templates; per-layer Kibana dashboard templates; the Jira scheme (work-type field, escaped-defect flag) and the ServiceNow-to-Jira-to-deployment linkage; Git webhook collection for PR analytics.

Each app team owns the semantics: defining and emitting activation events, workflow start/end/abandon events, and feature tags through the shared library; setting SLO targets and alert thresholds for their services; adding custom spans for business-critical operations and non-standard clients; linking incidents to changes, flagging escaped defects, and classifying work items; board hygiene and app-specific dormancy

thresholds. The platform makes collection free; it cannot make meaning free.

4. Layer 1: Product Analytics (What Users Do)

Behavioral and attitudinal signals captured through event instrumentation of the application suite. All events flow through the shared client event library into dedicated Elastic product-events indices, following a shared event taxonomy so metrics are comparable across applications.

#	Metric	Definition and Signal	Tracking and Calculation (Stack Implementation)	Owner
1	Activation rate	Percentage of new users who reach the defined 'aha moment' or complete a key first action.	App team defines and emits a named activation event via the shared library. Kibana Lens funnel from first-session event to activation event; rate = activated users / new users per acquisition cohort. Central: library, index, funnel template. App: activation definition and instrumentation.	Shared
2	Feature adoption	Percentage of users engaging with a given feature, plus time-to-first-use.	Feature-tagged events; terms aggregation of unique users per feature tag over active users. Time-to-first-use = first feature-event timestamp minus user first-seen (from the retention transform). Central: dashboards. App: feature tagging at entry points.	Shared
3	Retention (D1/D7/D30)	Percentage of users returning over time, measured by acquisition cohort.	Elastic transform pins first-seen date per user; scheduled queries count users with activity at D1/D7/D30 per cohort, segmented by application and role. Central: transform and queries. App: ensure every event carries the user and application dimensions.	Shared
4	Stickiness (DAU/MAU)	Ratio expressing how habitually the product is used.	Cardinality aggregation of daily vs. monthly unique users on the product-events index; ratio as a Lens formula per application. Fully automatic once events flow.	Central
5	Task success rate	Percentage of users completing core workflows without failure or abandonment.	App team defines core workflows and emits workflow start / complete / abandon events. Success = completes / starts per workflow per release, computed in Kibana. Central: event schema and dashboard template. App: workflow definitions and instrumentation.	Shared
6	Time to complete key tasks	Efficiency of critical flows; watch for regressions after releases.	Duration between workflow start and complete events; percentile aggregation per workflow, overlaid with deployment annotations so regressions pin to a release.	Shared
7	Error/friction rate	Rage clicks, dead clicks, form errors, and failed actions per session.	Elastic RUM agent captures JS errors; the shared library detects rage clicks, dead clicks, and form errors and emits friction events. Rate per session in Kibana; correlate with 4xx anomalies in Layer 2. Central: detection logic in the library. App: adopt the library.	Shared
8	Support contact rate	Tickets per active user, per application, and per workflow. Friction that escaped instrumentation.	Scheduled ServiceNow export into Elastic; tickets joined on application/workflow fields captured at intake, divided by active users from product events. Central: export and join. App/support: category discipline at ticket intake.	Shared

#	Metric	Definition and Signal	Tracking and Calculation (Stack Implementation)	Owner
9	Session depth/breadth	Actions per session and number of distinct features touched.	RUM session identifiers stamped onto product events; events per session and distinct feature tags per session via aggregations. Automatic once the library is adopted.	Central
10	User satisfaction	CSAT, SUS, or NPS tied to specific flows. Supplemented by support-derived satisfaction (ServiceNow CSAT, ticket sentiment), which samples only the unhappiest users.	Shared in-app micro-survey component fires at workflow completion and writes responses to Elastic; ServiceNow CSAT exported and reported as a separate series, never blended. Central: component and export. App: embed at the right workflow completions.	Shared
11	Churn signals/dormancy	Declining usage frequency before users go fully inactive.	Scheduled Elastic query computes last-activity per user; dormancy fires when the gap exceeds the app-defined threshold; cohort-level decline alerts route to product owners. Central: job and alerting. App: thresholds per application.	Shared

Where to start: Activation, retention, and task success carry the most diagnostic power with limited traffic. The remaining metrics layer in as baseline volume accumulates.

5. Layer 2: Technical Telemetry (How the Product Behaves)

Runtime reliability and performance signals, collected almost entirely through Elastic Observability. Because APM agents and Elastic Agent ship in the base images, most of this layer is instrumented by default; app teams contribute SLO targets, custom spans, and triage. Items 1 through 5 and 7 map closely to the golden signals model (latency, traffic, errors, saturation). Server and client error classes are tracked as separate metrics because they carry different meanings and different response models.

#	Metric	Definition and Signal	Tracking and Calculation (Stack Implementation)	Owner
1	Availability/uptime	Percentage of time the service is operational, measured against published SLOs.	Elastic Synthetics/Heartbeat monitors per service plus APM transaction success rate; formal Elastic SLO definitions with error budgets. Central: monitors and SLO tooling. App: SLO targets and critical user journeys to probe.	Shared
2	5xx server error rate	Unhandled exceptions, timeouts, crashed pods, exhausted pools, dead dependencies. Service failures; baseline near zero.	APM transaction outcomes filtered to 5xx per service/endpoint; SLO burn-rate alert rules page on-call through the ServiceNow alerting connector, which also stamps the detection timestamp for MTTD. Central: rule templates and routing. App: per-service thresholds.	Shared
3	4xx client error anomaly rate	Trended, not alerted; deltas correlated with deployments. Classes: 401/403 (auth and policy misconfiguration; a 403 spike after a policy change is an early warning of over-restriction), 404 (stale clients, API version mismatch), 400/422 (contract breaks between services), 429 (rate limiting).	APM HTTP status breakdowns per endpoint feed Elastic ML anomaly-detection jobs per class. The CI/CD pipeline posts APM deployment annotations on every deploy so anomaly deltas line up with releases and policy changes. Server-caused 4xx tagged at triage and fed to the Layer 3 escape rate. Central: ML jobs and annotations. App: triage of anomalies.	Shared
4	Crash rate	Unhandled client-side crashes per session.	Client crash handlers report through the RUM/APM agents; crashes per session segmented by app version, OS, and device from agent metadata. Central: ingestion and dashboards. App: wire crash handlers in non-web clients.	Shared
5	Latency (p50/p95/p99)	Response times at percentiles, never averages. Tail latency is where users feel pain.	APM agents in the base images record every transaction; percentile aggregations per endpoint and cohort in Kibana. Automatic; app teams add custom spans only for business-critical internal operations.	Central
6	Frontend performance	Core Web Vitals (LCP, INP, CLS) and time-to-interactive for web surfaces.	Elastic RUM agent (bundled in the shared client library) collects CWV and page-load timings into the Kibana UX dashboards. Central: snippet and dashboards. App: include the library.	Shared
7	Throughput	Requests and transactions per second; capacity headroom.	APM transaction rate per service against provisioned capacity from infrastructure metrics. Automatic.	Central
8	MTTD/MTTR	How fast incidents are detected and resolved.	ServiceNow incident records: detection timestamp (from the Elastic alert or manual report), opened, and resolved; computed in ServiceNow reporting and exported to Elastic for trend dashboards and postmortems.	Central

#	Metric	Definition and Signal	Tracking and Calculation (Stack Implementation)	Owner
9	Saturation/resource utilization	CPU, memory, storage, and connection pools under load.	Elastic Agent system and integration metrics fleet-wide; threshold alerts set well below hard limits. Automatic via the base images.	Central
10	API health per endpoint	Latency and 5xx/4xx breakdown by service and endpoint, never aggregate alone.	APM service and transaction views are already dimensioned by service, endpoint, and method; the dashboard template rolls up latency and error classes per endpoint. Automatic.	Central
11	Client-side telemetry	JS errors, ANRs, network failures, device and browser breakdowns.	RUM error and network data with user-agent breakdowns, correlated with backend releases via deployment annotations. Central: dashboards. App: library adoption on every client surface.	Shared
12	Dependency health	Third-party API latency and failures, database query performance, queue depths.	APM exit spans auto-capture outbound HTTP, database, and queue calls, giving downstream latency and failure rates; queue depth and broker health via Elastic Agent integrations. App teams add spans only for non-standard protocols.	Central

Alerting doctrine: Alert on 5xx, trend on 4xx. Server errors breach SLOs and page on-call engineers through the ServiceNow connector. Client-error anomalies are investigated during business hours, with auth failures as the exception. Expected 4xx (for example, a user mistyping a password) is excluded from SLO math.

Asynchronous and batch workloads

The metrics above are built around HTTP request/response. In a distributed architecture the asynchronous paths (batch jobs, queue consumers, scheduled pipelines, data syncs) are where failures hide longest, so they carry their own signals:

#	Metric	Definition and Signal	Tracking and Calculation (Stack Implementation)	Owner
1	Job success rate	Percentage of scheduled and batch runs completing successfully.	A shared job wrapper emits run start/end/outcome events to Elastic; success per job per day with alerts on consecutive failures of critical jobs. Central: wrapper and dashboards. App: adopt the wrapper in custom jobs.	Shared
2	Run duration drift	Growth in job runtime against its own baseline; the early warning before a job overruns its window.	Duration percentiles per job from the run events; Elastic ML anomaly job per critical pipeline.	Central
3	Queue age and depth	Oldest-message age per queue, not just depth. Depth can look flat while age explodes on a stuck consumer.	Elastic Agent broker integrations expose age and depth per queue; threshold alerts on age for critical flows.	Central
4	Dead-letter volume	Messages landing in dead-letter queues; each one is a silent processing failure.	DLQ arrival counts per queue from the broker integration; alert on nonzero for critical flows, trend elsewhere. App teams own triage and replay.	Shared

#	Metric	Definition and Signal	Tracking and Calculation (Stack Implementation)	Owner
5	Data freshness	Lag between a source update and the availability of derived data downstream.	Pipelines stamp a freshness watermark event on completion; freshness = now minus watermark, alerted against the per-dataset contract. Central: watermark convention. App: declare the contract per dataset.	Shared

Alert quality

The alerting doctrine says what pages; these metrics verify that paging is working. Alert fatigue quietly destroys MTTD, and converged teams arrive with very different alert hygiene:

#	Metric	Definition and Signal	Tracking and Calculation (Stack Implementation)	Owner
1	Pages per on-call week	Raw paging load per rotation; the fatigue signal.	ServiceNow paging records per rotation per team; trend against a sustainable ceiling.	Central
2	Actionable alert rate	Percentage of pages that required action; the precision of the alerting rules.	One-click triage tag at acknowledgment (actionable / noise / duplicate) in ServiceNow; rate per alert rule, with the noisiest rules reviewed monthly. Central: tagging mechanism. App: tagging discipline.	Shared
3	Auto-resolved alert rate	Alerts that close themselves before anyone acts; candidates for deletion or longer evaluation windows.	Elastic alert lifecycle data: fired-to-recovered without acknowledgment, per rule.	Central
4	Time to acknowledge	How fast pages are picked up; the human half of MTTD.	ServiceNow page-to-acknowledge timestamps, percentiles per rotation.	Central

6. Layer 3: Programmatic Delivery (How the Organization Delivers)

Flow and quality signals for the delivery system itself, computed from the CI/CD pipeline, Jira, and ServiceNow, with derived data landing in Elastic for dashboards. The four DORA metrics anchor this layer and live in a separate engineering-performance view, distinct from runtime telemetry. DORA's own research is explicit that these are system health indicators, not performance rankings; in a converged organization of this scale they become counterproductive quickly if read as team scorecards.

#	Metric	Definition and Signal	Tracking and Calculation (Stack Implementation)	Owner
1	Deployment frequency	How often code ships to production. (DORA)	A pipeline stage emits a deployment event (service, version, commit SHAs, timestamp, environment) to the Elastic deployments index on every production deploy; count per service per day. Zero per-team work once teams are on the golden pipeline.	Central
2	Lead time for changes	Commit to production; end-to-end pipeline speed. (DORA)	The deployment event carries the commit timestamps it ships; lead time = deploy time minus commit time, reported as percentiles per service.	Central
3	Change failure rate	Percentage of deployments causing incidents, rollbacks, or hotfixes. (DORA)	The pipeline auto-creates a ServiceNow change record per production deploy. Incidents, rollbacks, and hotfix deploys are linked to the originating change; CFR = failed changes / total deploys. Central: automation and linkage. App: discipline in linking incidents to the causing change.	Shared
4	Failed deployment recovery time	How fast a bad change is remediated. (DORA; replaced MTTR in the 2023 revision.)	Failure detection timestamp (Elastic alert or ServiceNow incident) to the remediation deployment event, joined in Elastic on the change record.	Central
5	PR cycle time	Open-to-merge duration, including time-to-first-review; the biggest hidden bottleneck in most organizations.	Git server webhooks (PR opened, first review, approved, merged) stream into an Elastic index; percentiles decomposed into wait states before anyone optimizes anything.	Central
6	Build/pipeline health	CI duration, flaky test rate, pipeline success rate.	The pipeline publishes stage durations and results per build; flaky test = pass-on-retry, tracked per suite and quarantined explicitly. Central: collection and flake detection. App: owning and fixing their flaky tests.	Shared
7	Test coverage & escape rate	Coverage as a floor; escapes as the real signal, arriving through monitoring, server-caused 4xx (Layer 2), and support tickets confirmed as defects.	Coverage from CI test reports published per build. Escapes = Jira defects carrying the escaped-to-production flag, auto-linked from ServiceNow tickets and tagged 4xx findings. Central: Jira scheme and linkage. App: triage tagging discipline.	Shared
8	Code churn/rework	Percentage of code rewritten shortly after merge; a proxy for unclear requirements or rushed work.	A scheduled analytics job over Git history computes lines rewritten within the rework window (e.g., 21 days) per repo, results indexed to Elastic.	Central
9	Work in progress & throughput	Items in flight versus completed per sprint; flow efficiency.	Jira board data via JQL export into Elastic; cumulative flow diagrams per team. Central: dashboards. App: board hygiene so statuses reflect reality.	Shared

#	Metric	Definition and Signal	Tracking and Calculation (Stack Implementation)	Owner
10	Work ratio	Distribution of effort across work types: new capability, debt remediation, defect fix, and keep-the-lights-on. A shrinking new-capability share is the early warning that debt and operational load are consuming the organization.	A mandatory Jira work-type field with the fixed four-value taxonomy; mix computed by JQL per organization and trended. Do not target a universal split. Central: field, taxonomy, report. App: honest classification.	Shared

Convergence caution: Baseline each legacy organization separately before setting any org-wide target. Starting points will differ widely, and a single early target punishes the teams that inherited the worst pipelines while teaching everyone to game the numbers.

7. Cross-Layer Linkages

The layers are only a strategy, rather than three disconnected dashboards, if they can be joined. The following linkages are first-class analysis paths and drive the shared-dimension requirement in Section 14. In this stack, every join happens in Elastic: product events, APM data, deployment events, and the ServiceNow and Jira exports all share the dimension schema.

Signal (Source Layer)	Connects To (Target Layer)	Question It Answers
p95+ latency and error rate, segmented by cohort (L2)	Friction rate, task success, churn signals (L1)	Is a slow or failing endpoint actually driving the friction and churn we observe?
4xx anomaly deltas after deployment (L2)	Change failure rate, escape rate (L3)	Which releases break client contracts, and is the failure rate improving?
Server-caused 4xx annotations (L2)	Escape rate (L3)	What defects are hiding in the client-error bucket?
Crash rate and client telemetry (L2)	Retention and satisfaction (L1)	Are stability problems on specific devices or versions costing us users?
Lead time and deployment frequency (L3)	Time-to-first-use of new features (L1)	Is delivery speed translating into user value, or just into deploys?
Support contact rate deltas after deployment (L1)	4xx anomalies, crash rate (L2); change failure rate (L3)	Did a release create friction that only surfaced through the human channel?
Support-detected vs. monitoring-detected incidents (L1 vs. L2)	MTTD (L2)	Do customers report incidents before monitoring does? This ratio is one of the most honest measures of telemetry maturity.

8. DORA Alignment

DORA (DevOps Research and Assessment) defines five metrics: four software delivery metrics established in the original research, plus reliability, added in 2021 to measure operational performance. This strategy adopts all five, but deliberately does not place them in a single list. The four delivery metrics measure how the organization ships change and anchor Layer 3; in this stack all four are computed centrally from pipeline deployment events and the ServiceNow change linkage, with no per-team collection work. The fifth measures how the product behaves in production, which is precisely what Layer 2 exists to capture; rather than appearing as one more row, it is decomposed into its constituent signals and summarized as Elastic SLO attainment.

DORA Metric	Measures	Location in This Strategy
1. Deployment frequency	How often code ships to production (velocity).	Layer 3, item 1.
2. Lead time for changes	Commit to production; end-to-end pipeline speed (velocity).	Layer 3, item 2.
3. Change failure rate	Percentage of deployments causing incidents, rollbacks, or hotfixes (stability).	Layer 3, item 3.

DORA Metric	Measures	Location in This Strategy
4. Failed deployment recovery time	How fast a bad change is remediated (stability). Replaced MTTR in the 2023 revision.	Layer 3, item 4.
5. Reliability	Operational performance: the ability to meet or exceed reliability targets, spanning availability, latency, performance, and scalability. Added in 2021.	Layer 2 in its entirety, summarized as SLO attainment. Availability (item 1), 5xx rate (item 2), latency percentiles (item 5), and saturation (item 9) are its constituent signals; MTTD/MTTR (item 8) and alerting doctrine cover the operational-practice dimension.

Why the split matters: DORA's own research found that teams at every level of delivery performance see better outcomes when they also prioritize operational performance. Keeping delivery metrics in Layer 3 and reliability in Layer 2, joined through the cross-layer linkages, makes that coupling explicit instead of flattening two distinct concerns into one list.

9. Customer Support Signals

Support metrics split into two categories, and the distinction determines where they live. Signals about the product flow into the existing layers, because a support ticket is user friction that escaped instrumentation. Metrics about the support function itself form a companion operational view, kept outside the three layers for the same reason DORA lives in its own engineering-performance view: a different system under measurement, a different owner, and distinct failure modes if conflated. ServiceNow is the system of record throughout; the companion view is computed in ServiceNow reporting, with the product-facing signals exported to Elastic for joins.

Product-facing support signals (folded into the layers)

Support contact rate (Layer 1, item 8). Tickets per active user, application, and workflow. Sits beside error/friction rate and catches what behavioral analytics miss, since users who hit a wall and contact support never generate the in-product events.

Ticket-confirmed defects (Layer 3, escape rate). A ticket that traces to a shipped defect is an escaped defect, joining monitoring-detected escapes and server-caused 4xx as the third detection channel. Implemented as the ServiceNow-to-Jira linkage with the escaped-defect flag.

Release-correlated ticket spikes (annotation discipline). A step change in contact rate after a deployment is the same class of signal as a 4xx anomaly delta, arriving through a human channel with more latency. The exported ticket series carries the same deployment annotations in Kibana.

Support-derived satisfaction (Layer 1, item 10). ServiceNow CSAT on resolved tickets and ticket sentiment supplement in-product satisfaction, reported separately because they sample only the unhappiest users.

Support operations companion view

These measure how effectively the support function delivers, owned by the support lead and reviewed on its own cadence. All are computed natively in ServiceNow reporting (Central), except deflection, which joins Elastic help-content events to subsequent ticket creation (Shared):

Metric	Definition and Signal	Tracking and Calculation	Owner
First response time	Time from ticket creation to first substantive human response.	ServiceNow timestamps; percentiles, not averages, per priority tier.	Central
Resolution time	Time from creation to confirmed resolution.	ServiceNow SLA clocks exclude time waiting on the customer; segment by ticket category.	Central
Backlog & aging	Open tickets by age band; growth or drawdown of the queue.	ServiceNow reporting, trended weekly; the aging distribution matters more than raw count.	Central
First-contact resolution	Percentage of tickets resolved without escalation or reopening.	ServiceNow reopen tracking within a defined window counts as failure.	Central
Escalation rate	Percentage of tickets escalated to engineering; a load signal on delivery teams.	ServiceNow assignment-group transitions tagged by receiving team; feeds the keep-the-lights-on share of the Layer 3 work ratio via the Jira linkage.	Central
Deflection rate	Percentage of contacts resolved by self-service (documentation, in-product help) before reaching a human.	Help-content usage events (shared library, Elastic) joined to subsequent ticket creation per user within a window. Central: join. App: instrument help surfaces.	Shared

The observability honesty check: Track the ratio of support-detected to monitoring-detected incidents (ServiceNow incidents opened manually vs. opened by the Elastic alerting connector). If customers routinely report incidents before monitoring does, the telemetry has a gap; this ratio is one of the most honest measures of observability maturity available to engineering leadership.

10. Measurement Integrity

Nothing in the preceding sections watches the watchers. A silently broken event stream looks identical to good news: friction drops to zero, contact rate falls, dashboards go green. This section protects the credibility of every other metric in the framework, and its signals go live in Phase 1, before anything they guard.

Instrumentation health

#	Metric	Definition and Signal	Tracking and Calculation (Stack Implementation)	Owner
1	Event volume anomaly	A step drop in event volume per application or event type is a broken stream, not an improvement.	Elastic ML anomaly job on event counts per application, event type, and release version; a drop correlated with a deployment pages the owning team the same way a 5xx burn does.	Central
2	Schema validation failure rate	Events rejected or coerced by the ingest pipelines; drift between the client library and the index templates.	Ingest pipeline failure processors route rejects to a quarantine index with the violation reason; rate per application trended, spikes investigated before the data is trusted.	Central
3	Export freshness	Lag of the ServiceNow and Jira exports behind their sources; stale joins silently skew every cross-system metric.	Each export run stamps a watermark; freshness alerted against the export schedule contract.	Central

Metric definitions registry

Definitions of 'active user', 'session', 'core workflow', and 'incident' will drift across converged organizations, and two dashboards showing different retention numbers for the same application destroys trust in the whole program. The registry is a versioned data dictionary, held alongside the dashboards: one entry per defined term, a named owner per definition, the exact query or event logic that implements it, and a change history. A definition change is treated like a deployment: it requires the owner's approval and it annotates every chart it touches, because a definitional step change is indistinguishable from a real one otherwise. Central: the registry and the annotation mechanism. App: ownership of their application-specific definitions.

Program adoption

The rollout is itself measured. These are platform engineering's own success metrics and the honest predictor of whether Phases 2 and 3 land on schedule:

#	Metric	Definition and Signal	Tracking and Calculation (Stack Implementation)	Owner
1	Golden pipeline adoption	Percentage of services deploying through the golden pipeline.	Deployment events carry a pipeline identifier; services deploying outside the paved road are visible by omission.	Central
2	Event library adoption	Percentage of applications emitting through the shared client library, and version skew across them.	Library version reported in event metadata; adoption and skew per application.	Central

#	Metric	Definition and Signal	Tracking and Calculation (Stack Implementation)	Owner
3	Change-record linkage rate	Percentage of production deploys with a linked ServiceNow change record.	Join of the deployments index to change records; gaps mean the DORA metrics under-count.	Central
4	Classification fill rate	Percentage of Jira items carrying the work-type field and of resolved defects with the escape flag adjudicated.	JQL reports per organization; low fill rate disqualifies that org's work ratio from review until fixed.	Central

11. Perceptual Measures

Every metric so far is system-derived, and system data alone misleads: the SPACE research that followed DORA is explicit that flow metrics without perceptual data cannot distinguish real improvement from gamed behavior. An improving PR cycle time means something different when developers report rising friction than when they report falling friction. Perceptual measures are the control group for the rest of the framework.

Developer experience survey. A short quarterly instrument covering friction, tooling satisfaction, cognitive load, and confidence in the delivery process, run on self-hosted survey tooling with results indexed to Elastic alongside the Layer 3 trends. Reported per organization against its own prior quarters, never ranked across organizations, and read side by side with the flow metrics: divergence between the two is the finding. Central: the instrument and analysis. Leadership: acting on it visibly, because a survey nobody acts on trains people to stop answering honestly.

User perception panel. A small standing panel of real users engaged by the UX research function on a recurring cadence, complementing the in-app micro-surveys with depth the event stream cannot carry: why tasks feel hard, what workarounds exist, what the contact-rate numbers mean in human terms. Findings are attached to the Layer 1 review as qualitative context, not converted into a score.

12. Cost and Capacity Efficiency

The framework excludes revenue, not cost. Cost is an engineering signal: it reflects architectural efficiency and it constrains everything else, including this strategy's own data footprint.

#	Metric	Definition and Signal	Tracking and Calculation (Stack Implementation)	Owner
1	Cost per service	Infrastructure cost allocated by service, trended as cost per transaction so growth in spend is distinguishable from growth in load.	Resource tagging by service enforced at provisioning; allocation joined in Elastic to APM throughput for the per-transaction view. Central: allocation and dashboards. App: tagging compliance.	Shared
2	Utilization efficiency	Provisioned versus consumed capacity; sustained low utilization is spend without headroom benefit.	Derived from the Layer 2 saturation data against provisioned capacity; right-sizing candidates surfaced quarterly.	Central
3	Observability footprint	Storage and ingest trajectory of the telemetry platform itself, which this strategy grows aggressively.	Elastic index lifecycle management (hot/warm/cold tiers and retention windows) defined per index class (APM, product events, derived); cluster storage and ingest rate trended with projections reviewed quarterly, before the retention bill is discovered the hard way.	Central

13. Operating Rhythm and Decision Rights

Review cadences without decision rights produce reporting, not management. Each trigger below names the decision it forces and who holds it; reviews exist to take these decisions, and a review that takes none records why. Thresholds are set per organization against Phase 1 baselines, and contested calls escalate one level, once.

Trigger	Decision Forced	Decision Owner
Error budget exhausted on a service SLO	Feature releases for that service pause in favor of reliability work until the budget recovers.	Service owner with SRE; engineering leadership arbitrates if contested.
Confirmed 4xx anomaly or crash spike after a deployment	Roll back or hotfix within the business day; the change is linked as failed either way.	Owning app team lead.
Support-detected incidents exceed monitoring-detected for a service	Observability gap review; new monitors or SLIs committed for that service.	Platform engineering with the service owner.
New-capability share of the work ratio below the org's baseline band for two consecutive quarters	Debt investment review; explicit rebalancing of the next quarter's capacity.	Engineering leadership with product.
Step change in change failure rate or escape rate for an organization	Delivery retrospective on the affected pipeline or review practice.	That organization's engineering manager.
Alert rule below the actionable-rate floor	Tune, re-scope, or delete the rule; noisy rules do not survive by default.	SRE with the owning team.
Metric definition change request	Approve and version in the registry; annotation applied to every affected chart.	The definition's named owner.
Instrumentation health alarm (volume drop, schema failures, stale export)	Affected metrics flagged untrusted until the stream is repaired; no decisions taken on them in the interim.	Platform engineering.

14. Collection Requirements

Shared dimensions. Every event and metric carries release version, service, endpoint (where applicable), and an anonymized user-cohort identifier, enforced by the Elastic index templates and ingest pipelines. Cross-layer joins depend on this and it cannot be retrofitted cheaply.

A common event taxonomy. Product events follow one naming convention across the application suite, carried by the shared client library, so Layer 1 metrics are comparable between applications from day one.

Deployment and policy-change annotations. Every dashboard that trends deltas overlays the APM deployment annotations posted by the pipeline, plus authorization policy changes, because step changes correlated with a change event are the primary actionable signal.

Percentiles over averages. Latency and duration metrics are collected as histograms and reported as percentiles.

SLO discipline. Availability, 5xx rate, and latency carry published Elastic SLOs with error budgets. Nothing else pages.

System-of-record discipline. Jira for work items, ServiceNow for incidents and changes, the pipeline for deployments, Elastic for everything derived. No metric is hand-maintained in a spreadsheet; if it cannot be computed from a system of record, it is not in the framework.

Data lifecycle discipline. Every index class carries an ILM policy (tiers and retention window) from the day it is created, sized to the review cadence that reads it; telemetry without a declared lifecycle is not ingested.

Privacy and least data. Behavioral analytics use anonymized or pseudonymized identifiers; collect the minimum needed to answer the defined questions.

15. Rollout Sequence

Each phase names the metrics it stands up and the platform deliverable that makes them possible. The sequencing logic: Phase 1 is everything that comes free or nearly free from the paved road plus the three highest-diagnostic-power product metrics, with the measurement-integrity signals live from day one, before anything they guard; Phase 2 is everything that requires segmentation, ML jobs, or the support integration; Phase 3 is everything that requires sustained classification discipline from teams, which only sticks once the earlier habits exist.

Phase 1 (Quarters 1-2): Instrument the floor

Platform engineering ships the paved road: APM and Elastic Agent in base images, the deployment-event and annotation pipeline stages, automated ServiceNow change records, the alerting connector, and SLO tooling on critical services. App teams onboard to the golden pipeline and define their first activation and workflow events. Capture per-organization baselines. No targets yet.

Layer / View	Metrics Onboarded	Enabling Platform Deliverable
Layer 1	Activation rate (1), Retention D1/D7/D30 (3), Task success rate (5)	Shared client event library and product-events index; app teams define activation and workflow events on flagship applications

Layer / View	Metrics Onboarded	Enabling Platform Deliverable
Layer 2	Availability/uptime (1), 5xx error rate (2), Latency p50/p95/p99 (5), Throughput (7), MTTD/MTTR (8), Saturation (9), API health per endpoint (10)	APM and Elastic Agent in base images; Synthetics monitors; SLO definitions with burn-rate alerting routed through the ServiceNow connector
Layer 3	Deployment frequency (1), Lead time for changes (2), Change failure rate (3), Failed deployment recovery time (4)	Pipeline deployment events to the Elastic deployments index; automated ServiceNow change records and incident linkage. All four compute with zero per-team work
Integrity (S10)	Event volume anomaly, Schema validation failure rate; Golden pipeline adoption, Event library adoption, Change-record linkage rate; the definitions registry seeded with the first terms	ML jobs and quarantine indices built with the ingest pipelines; adoption computed from event metadata. The rollout is measured by its own adoption metrics from day one

Phase 2 (Quarters 2-3): Segment and correlate

Everything that turns raw collection into diagnosis: per-endpoint segmentation, anomaly detection against deployment annotations, full client-side visibility, the remaining behavioral metrics, and the support integration. Establish the cross-layer joins in Section 7, including the observability honesty check.

Layer / View	Metrics Onboarded	Enabling Platform Deliverable
Layer 1	Feature adoption (2), Stickiness DAU/MAU (4), Time to complete key tasks (6), Error/friction rate (7), Support contact rate (8), Session depth/breadth (9), User satisfaction (10), Churn signals/dormancy (11)	Friction detection and in-app survey component in the shared library; RUM session stamping; dormancy jobs; scheduled ServiceNow ticket export joined to product events
Layer 2	4xx anomaly rate (3), Crash rate (4), Frontend performance (6), Client-side telemetry (11), Dependency health (12)	ML anomaly jobs per 4xx class overlaid with deployment annotations; RUM rollout to every client surface; exit-span dashboards and queue integrations
Support view	First response time, Resolution time, Backlog and aging, First-contact resolution, Escalation rate; the support-detected vs. monitoring-detected incident ratio	ServiceNow reporting configuration; assignment-group tagging; the Elastic alerting connector distinguishing auto-opened from manually opened incidents
Extensions (S5, S10-12)	Async/batch workload signals (job success, duration drift, queue age, dead-letter volume, data freshness); Alert quality (pages per week, actionable rate, auto-resolved rate, time to acknowledge); Export freshness; Observability footprint and ILM; first developer experience survey as the baseline	Job wrapper and freshness watermarks; triage tagging at acknowledgment; ILM policies per index class; self-hosted survey instrument stood up

Phase 3 (Quarter 4 onward): Mature the delivery view

The remaining Layer 3 metrics all depend on sustained team discipline (classification, triage tagging, board hygiene), which is why they come last: the habits have to exist before the data is trustworthy. Set improvement goals per organization against their own Phase 1 baselines, reviewed quarterly. Only then consider org-wide reference ranges.

Layer / View	Metrics Onboarded	Enabling Platform Deliverable
Layer 3	PR cycle time (5), Build/pipeline health (6), Test coverage and escape rate (7), Code churn/rework (8), WIP and throughput (9), Work ratio (10)	Git webhook collection; pipeline stage telemetry and flake detection; the Jira scheme (work-type field, escaped-defect flag) with the ServiceNow-to-Jira linkage; churn analytics job; JQL exports
Support view	Deflection rate	Help-content usage events from the shared library joined to subsequent ticket creation; requires app teams to have instrumented help surfaces
Operating model (S11-13)	Cost per service and utilization efficiency reviews; quarterly developer experience survey trended against the now-live flow metrics; user perception panel standing; decision-rights thresholds (Section 13) activated per organization	Service cost allocation joined to APM throughput; survey cadence established; thresholds set only now, against each organization's own Phase 1 baselines

Summary: product analytics tell us what users do, technical telemetry tells us how the product behaves, and programmatic metrics tell us how the organization delivers. Implemented on Elastic, the CI/CD pipeline, Jira, and ServiceNow, collected with shared dimensions, and read as trends against per-organization baselines, the three layers form a single defensible measurement architecture rather than three competing dashboards. Platform engineering makes collection free; app teams supply the meaning.